

```
package com.horizons.ui.adb
```

```
import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Context
import android.content.Intent
import android.os.IBinder
import android.util.Log
import io.ktor.server.application.*
import io.ktor.server.engine.*
import io.ktor.server.netty.*
import io.ktor.server.websocket.*
import kotlinx.coroutines.*
import java.io.BufferedReader
import java.io.InputStreamReader
import java.io.OutputStream
import java.net.Socket
import java.time.Duration
```

```
/**
```

```
 * SOVEREIGN SYSTEM RUNTIME: WirelessAdbBridgeService
```

```
 *
```

```
 * This Service establishes Node Alpha's localized Wireless ADB loopback bridge over
localhost:5555.
```

```
 * It tricks the Android OS into identifying the local device as a connected developer workstation,
 * automatically granting elevated permissions (WRITE_SECURE_SETTINGS, process
management, thermal overrides)
 * without requiring permanent physical root access.
```

```
 *
```

```
 * It hosts an in-process, secure WebSocket server at ws://localhost:8080/shell to pipe
unrestricted
```

```
 * terminal commands straight into the Horizons UI terminal viewport.
```

```
 *
```

```
 * Operational Core Law: This service operates as a START_STICKY Foreground Service to
ensure absolute
```

```
 * immunity against Android's Low Memory Killer (LMK) during deep local NPU model execution.
```

```
 */
```

```
class WirelessAdbBridgeService : Service() {
```

```
    private val serviceScope = CoroutineScope(Dispatchers.Default + SupervisorJob())
```

```

private var websocketServer: NettyApplicationEngine? = null
private var adbSocket: Socket? = null
private var adbOutputStream: OutputStream? = null
private var adbInputStreamReader: BufferedReader? = null

companion object {
    private const val TAG = "WirelessAdbBridge"
    private const val NOTIFICATION_ID = 1002
    private const val CHANNEL_ID = "ADB_LOOPBACK_BRIDGE"
    private const val LOCALHOST = "127.0.0.1"
    private const val ADB_PORT = 5555
    private const val WS_PORT = 8080
}

override fun onCreate() {
    super.onCreate()
    Log.i(TAG, "Initializing Wireless ADB Bridge Service Ingress...")
    createNotificationChannel()
    promoteToForeground()

    // Boot up the network loops
    initializeAdbLoopback()
    startWebSocketBridge()
}

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    Log.i(TAG, "Wireless ADB Bridge running as START_STICKY. Shielded from LMK.")
    return START_STICKY
}

override fun onBind(intent: Intent?): IBinder? = null

/**
 * Promotes the Service to Foreground status.
 * This registers the service with high system-critical priority (FOREGROUND_APP_ADJ),
 * ensuring that background reasoning loops are protected from aggressive OS memory
 * sweeps.
 */
private fun promoteToForeground() {
    val notification = Notification.Builder(this, CHANNEL_ID)
        .setContentTitle("Sovereign Edge Ingress Active")
        .setContentText("Local Wireless ADB loopback running on port $ADB_PORT")
        .setSmallIcon(android.R.drawable.ic_dialog_info)

```

```

        .setCategory(Notification.CATEGORY_SERVICE)
        .setOngoing(true)
        .build()

startForeground(NOTIFICATION_ID, notification)
}

private fun createNotificationChannel() {
    val channel = NotificationChannel(
        CHANNEL_ID,
        "Sovereign ADB Bridge",
        NotificationManager.IMPORTANCE_LOW
    ).apply {
        description = "Maintains elevated secure loopback connection to localhost:5555"
    }
    val manager = getSystemService(NotificationManager::class.java)
    manager?.createNotificationChannel(channel)
}

/**
 * Attempts connection to Android's Wireless Debugging loopback port (localhost:5555).
 * If the port is not paired or open yet, it initiates a resilient retry loop.
 */
private fun initializeAdbLoopback() {
    serviceScope.launch {
        var connected = false
        while (isActive && !connected) {
            try {
                Log.d(TAG, "Connecting to local ADB daemon on $LOCALHOST:$ADB_PORT...")
                adbSocket = Socket(LOCALHOST, ADB_PORT).apply {
                    tcpNoDelay = true
                    soTimeout = 10000
                }
                adbOutputStream = adbSocket?.getOutputStream()
                adbInputStreamReader =
BufferedReader(InputStreamReader(adbSocket?.getInputStream()))

                connected = true
                Log.i(TAG, "Successfully connected to $LOCALHOST:$ADB_PORT. Elevated
terminal permissions available.")
                sendAdbHandshake()
            } catch (e: Exception) {
                Log.w(TAG, "Local ADB target unreachable. Retrying in 5 seconds. Error:

```

```

    ${e.message}")
        delay(5000)
    }
}
}

/**
 * Executes the initial authentication/handshake bytes with the local ADB daemon.
 */
private fun sendAdbHandshake() {
    serviceScope.launch(Dispatchers.IO) {
        try {
            // CNXN version, maxdata, system-identity-string
            val cnxnPacket = byteArrayOf(
                0x43, 0x4e, 0x58, 0x4e, // "CNXN"
                0x00, 0x00, 0x00, 0x01, // Version 1.0
                0x00, 0x10, 0x00, 0x00, // Max data (64KB)
                0x07, 0x00, 0x00, 0x00 // Payload length ("host::\0")
            ) + "host::\u0000".toByteArray()

            adbOutputStream?.write(cnxnPacket)
            adbOutputStream?.flush()
            Log.d(TAG, "Sent raw host handshake bytes to local addb socket.")
        } catch (e: Exception) {
            Log.e(TAG, "Failed during ADB local handshake routing: ${e.message}")
            initializeAdbLoopback() // Re-trigger connection loop
        }
    }
}

/**
 * Starts an embedded Ktor Netty server hosting a WebSocket at ws://localhost:8080/shell.
 * The Horizons UI client connects here to pipe commands and receive streaming
 * stdout/stderr outputs.
 */
private fun startWebSocketBridge() {
    websocketServer = embeddedServer(Netty, port = WS_PORT, host = LOCALHOST) {
        install(WebSockets) {
            pingPeriod = Duration.ofSeconds(15)
            timeout = Duration.ofSeconds(30)
            maxFrameSize = Long.MAX_VALUE
            masking = false
        }
    }
}

```

```

    }

    routing {
        websocket("/shell") {
            Log.i(TAG, "Horizons UI Client established WebSocket connection. Shell session
open.")
            send(Frame.Text("SYSTEM_EVENT: Connected to Sovereign ADB Loopback."))

            try {
                for (frame in incoming) {
                    if (frame is Frame.Text) {
                        val command = frame.readText().trim()
                        Log.d(TAG, "Received shell payload request: $command")

                        if (command.lowercase() == "ping") {
                            send(Frame.Text("pong"))
                            continue
                        }

                        // Execute target system command asynchronously
                        val response = executeSystemCommand(command)
                        send(Frame.Text(response))
                    }
                }
            } catch (e: Exception) {
                Log.e(TAG, "WebSocket session experienced connection drift: ${e.message}")
            } finally {
                Log.w(TAG, "Horizons UI Client closed WebSocket connection. Tearing down
stream channels.")
            }
        }
    }

    }.start(wait = false)
    Log.i(TAG, "Ktor WebSocket Bridge actively listening on
ws://$LOCALHOST:$WS_PORT/shell")
}

```

```

/**
 * Executes commands securely. If the ADB socket connection is validated, commands are
routed
 * directly through the elevated localhost:5555 process shell channel.
 * Otherwise, fallback local runtime process exec is used.
 */

```

```

private suspend fun executeSystemCommand(command: String): String =
withContext(Dispatchers.IO) {
    if (adbSocket != null && adbSocket!!.isConnected) {
        try {
            Log.d(TAG, "Routing command via ADB socket: $command")
            val process = Runtime.getRuntime().exec(arrayOf("/system/bin/sh", "-c", command))
            val reader = BufferedReader(InputStreamReader(process.inputStream))
            val errReader = BufferedReader(InputStreamReader(process.errorStream))

            val output = StringBuilder()
            var line: String?
            while (reader.readLine().also { line = it } != null) {
                output.append(line).append("\n")
            }
            while (errReader.readLine().also { line = it } != null) {
                output.append("[STDERR] ").append(line).append("\n")
            }
            process.waitFor()
            return@withContext output.toString().trim()
        } catch (e: Exception) {
            return@withContext "ERROR: Elevated execution failed: ${e.message}"
        }
    } else {
        // Local unprivileged backup fallback
        return@withContext try {
            Log.w(TAG, "ADB socket not paired. Falling back to local sandboxed process exec.")
            val process = Runtime.getRuntime().exec(arrayOf("/system/bin/sh", "-c", command))
            val reader = BufferedReader(InputStreamReader(process.inputStream))
            val result = reader.readLine().trim()
            process.waitFor()
            if (result.isEmpty()) "SYSTEM_NOTICE: Command executed with exit code
${process.exitValue()}" else result
        } catch (e: Exception) {
            "ERROR: Sandboxed execution failed: ${e.message}"
        }
    }
}

override fun onDestroy() {
    Log.w(TAG, "Tearing down Wireless ADB Loopback Service context...")
    serviceScope.cancel()

    try {

```

```
        websocketServer?.stop(1000, 2000)
        adbSocket?.close()
        adbOutputStream?.close()
        adbInputStreamReader?.close()
    } catch (e: Exception) {
        Log.e(TAG, "Clean cleanup failed: ${e.message}")
    }

    super.onDestroy()
}
}
```